C# / .NET STUDY NOTES

Value Types, Reference Types, Boxing and Unboxing in C#

Roadmap Topic 6

Learning focus
Value and reference semantics, copying, object mutation, structs/classes, stack and heap concepts, boxing, unboxing, equality, and performance.

Examples
Gift entities, Money values, collections, EF Core identity concepts, DI shared state, boxing costs, and safe copying behavior.

Supplied study notes → structured sections → examples and flows → revision and interview preparation

Contents

Click a section name to jump directly to that part of the notes.

01 Topic Coverage
02 Why This Topic Is Important
03 The Two Main Type Categories
04 What Is a Value Type?
05 Definition
06 Value Type Copy Example
07 Value Type Copy Flow
08 Common Value Types
09 Fundraising Value Type Examples
10 What Is a Reference Type?
11 Definition
12 Reference Type Example
13 Modifying Through One Reference
14 Reference Copy Flow
15 Common Reference Types
16 Fundraising Reference Type Examples
17 Value Type vs Reference Type — Basic Comparison
18 Value Semantics
19 Reference Semantics
20 Important Distinction: Variable vs Object
21 Reassigning a Reference Variable
22 Object Mutation vs Reference Reassignment
23 Object Mutation
24 Reference Reassignment
25 Reference Reassignment Flow
26 struct Is a Value Type
27 Struct Copy Example
28 Struct Copy Flow
29 But Structs Can Contain References
30 Struct Containing Reference Flow
31 Important Rule
32 enum Is a Value Type
33 Records and Type Category
34 Arrays Are Reference Types
35 Array Reference Flow
36 Copying an Array Reference Is Not Copying Array Ele...
37 Lists Are Reference Types
38 string Is a Reference Type
39 String Example
40 String Immutability Flow
41 object Is a Reference Type
42 Interfaces Are Reference Types
43 Value Types Can Implement Interfaces
44 Delegates Are Reference Types
45 Value Type Assignment
46 Reference Type Assignment
47 Copying a Class Reference Does Not Clone the Object
48 Shallow Copy vs Deep Copy
49 Shallow Copy
50 Deep Copy
51 Value Type Parameters — Review
52 Reference Type Parameters — Review

53 Reference Parameter Reassignment
54 ref Changes the Variable Relationship
55 Stack and Heap — Important Warning
56 What Is the Stack?
57 Stack Frame Flow
58 What Is the Managed Heap?
59 Why “Value Type = Stack” Is Wrong
60 Value Type Inside an Object
61 Value Types Inside Arrays
62 Reference Variables Can Also Exist in Objects
63 Better Stack/Heap Explanation
64 Runtime Optimizations
65 What Is Boxing?
66 Definition
67 Boxing Flow
68 Boxing Creates a Copy
69 Boxing Copy Flow
70 Why Boxing Exists
71 Boxing Is Usually Implicit
72 More Boxing Examples
73 Boxing to an Interface
74 Why Interface Boxing Matters
75 What Is Unboxing?
76 Definition
77 Unboxing Is Explicit
78 Unboxing Runtime Check
79 Exact Type Requirement
80 Correct int-to-long After Boxing
81 Invalid Unboxing Example
82 Unboxing null
83 Safer Type Checking
84 Boxing and object Collections
85 Generic Collections Avoid Much Boxing
86 Old Collection vs Generic Collection
87 Boxing with object[]
88 Boxing Can Be Hidden
89 Boxing Through Interfaces
90 Generics Help Preserve Value Types
91 Boxing Performance
92 Boxing Cost Flow
93 Example of Repeated Boxing
94 Better Generic/Typed Approach
95 Do Not Fear Boxing Everywhere
96 Nullable Value Types — Boxing Behavior
97 Nullable Boxing with Value
98 Nullable Boxing with null
99 Nullable Unboxing
100 Equality — Value Types
101 Struct Equality
102 Equality — Reference Types
103 ReferenceEquals
104 == for Classes
105 Strings Use Value Equality
106 Records Use Value-Oriented Equality

107 Value Type Does Not Mean Immutable
108 Mutable Struct Problems
109 Why Immutable Structs Are Often Preferred
110 readonly struct
111 Class vs Struct — Beginner Rule
112 Fundraising Example — Gift Should Usually Be a Class
113 Fundraising Example — Money Could Be a Value Object
114 Entity vs Value Concept
115 Large Structs and Copying
116 in Parameter for Large Structs
117 ref Returns and ref Locals — Preview
118 Value Type in List
119 Struct in List
120 Mutable Struct in Collection Problem
121 Reference Type in List
122 Collection Comparison
123 Class Field Containing a Struct
124 Struct Containing a Class Reference
125 Reference Cycles
126 Null and Reference Types
127 Value Type Default
128 Reference Type Default
129 Copy Semantics in Fundraising Platform
130 Compare with Struct Fundraising Value
131 Reference Identity in Fundraising
132 Separate Objects with Same Data
133 Why This Matters in EF Core Later
134 Why This Matters in Dependency Injection
135 Why This Matters in Async and Concurrency
136 Why This Matters in ASP.NET Core
137 Common Beginner Mistake — “Classes Are Passed by Re...
138 Common Beginner Mistake — “Value Types Are Always o...
139 Common Beginner Mistake — “Reference Types Are Alwa...
140 Common Beginner Mistake — Thinking Assignment Clone...
141 Common Beginner Mistake — Thinking Struct Copy Is D...
142 Common Beginner Mistake — Wrong Unboxing Type
143 Common Beginner Mistake — Assuming Casting and Unbo...
144 Common Beginner Mistake — Using object When Generic...
145 Common Beginner Mistake — Mutable Struct for Entity
146 Common Beginner Mistake — Using class for Every Tin...
147 Performance Considerations
148 Value Copy Cost
149 Large Struct Copy Cost
150 Boxing Allocation Cost
151 Unboxing Cost
152 Generics and Performance
153 Avoid Premature Struct Optimization
154 Security and Correctness Considerations
155 Shared Mutable State
156 Immutability Helps
157 Practical Exercise — Value Copy
158 Practical Exercise — Reference Copy

159 Practical Exercise — Reference Reassignment
160 Practical Exercise — Array Reference
161 Practical Exercise — String
162 Practical Exercise — Boxing
163 Practical Exercise — Valid Unboxing
164 Practical Exercise — Invalid Unboxing
165 Practical Exercise — Correct Numeric Conversion Aft...
166 Practical Exercise — Struct Copy
167 Practical Exercise — Struct with Reference
168 Interview Questions
169 What is the difference between value types and refe...
170 Give examples of value types.
171 Give examples of reference types.
172 What happens when you assign one value-type variabl...
173 What happens when you assign one reference-type var...
174 Does assigning one class variable to another clone...
175 What is the difference between mutating an object a...
176 Are reference types passed by reference to methods...
177 Are value types always stored on the stack?
178 Are reference types always stored on the heap?
179 What is boxing?
180 Is boxing implicit or explicit?
181 What is unboxing?
182 Why is unboxing explicit?
183 What happens if you unbox to the wrong type?
184 Can a boxed int be directly unboxed as long?
185 What happens when null is unboxed to a non-nullable...
186 Does boxing create a copy?
187 Can boxing occur when converting to an interface?
188 Why can boxing hurt performance?
189 How do generics help avoid boxing?
190 Is string a value type?
191 Are arrays value types?
192 Is a struct a value type?
193 Is a record a value type or reference type?
194 Is an enum a value type?
195 Can a struct contain a reference type?
196 Why are mutable structs often discouraged?
197 When should I choose a class instead of a struct?
198 When should I consider a struct?
199 What is the difference between ReferenceEquals and...
200 Best Practices
201 Quick Revision
202 Complete Fundraising Example
203 Complete Concept Flow

Topic Coverage

This is one of the most important C# topics for interviews and for understanding how objects, parameters, collections, generics, equality, memory, and performance work.

You already learned that C# variables have types.

Now we need to understand the two major categories of C# types:

```
C# Types
    →
Value Types
    +
Reference Types
```

This topic covers:

- what a value type is;
- what a reference type is;
- built-in value types;
- built-in reference types;
- `struct`;
- `enum`;
- `class`;
- arrays;
- `string`;
- `object`;
- interfaces and delegates;
- how assignment works;
- value copying;
- reference copying;
- how multiple references can point to one object;
- object mutation;
- reference reassignment;
- passing value types to methods;
- passing reference types to methods;
- stack and heap concepts;
- why “value type = stack” is not a correct universal rule;
- managed heap;
- method stack frames;
- boxing;
- unboxing;
- boxing to `object`;
- boxing to interfaces;
- exact-type rules for unboxing;
- invalid unboxing;

- nullable value-type boxing;
- performance cost of boxing;
- how generics reduce boxing;
- equality basics;
- mutable structs;
- choosing class vs struct;
- Fundraising Platform examples;
- common mistakes;
- practical exercises;
- interview questions;
- beginner-to-advanced flow.

The central idea is:

```
Value Type Variable
    →
contains its value

Reference Type Variable
    →
contains a reference
    →
reference points to an object
```

Microsoft describes C# the same fundamental way: value-type variables directly contain their data, while reference-type variables hold references to objects.

Why This Topic Is Important

Consider:

```
int first = 10;

int second = first;

second = 20;
```

What is `first` now?

```
10
```

But consider:

```
Gift first = new Gift();

Gift second = first;

second.TotalAmount = 5000m;
```

What does `first.TotalAmount` contain?

Potentially:

```
5000
```

Why did changing `second` affect something visible through `first`?

The reason is the difference between:

```
copying a value
```

and:

```
copying a reference
```

This distinction affects almost everything you later learn:

```
Methods
    →
OOP
    →
Collections
    →
Generics
    →
LINQ
    →
Async programming
    →
Dependency Injection
    →
EF Core
    →
ASP.NET Core
```

The Two Main Type Categories

C# has two primary type categories:

```
Value Types
Reference Types
```

The language specification also includes pointer/function-pointer concepts for specialized scenarios, but for normal managed C# application development the main distinction you must master is:

```
Value
vs
Reference
```

Microsoft's C# type system explicitly separates value types from reference types.

What Is a Value Type?

Definition

A **value type variable directly contains its value**.

Example:

```
int giftCount = 10;
```

Conceptually:

```
giftCount
    →
10
```

The variable itself represents that value.

If you copy it:

```
int anotherCount = giftCount;
```

the value is copied.

Now conceptually:

```
giftCount
    →
10

anotherCount
    →
10
```

These are separate values.

Value Type Copy Example

```
int first = 10;

int second = first;

second = 50;

Console.WriteLine(first);
Console.WriteLine(second);
```

Output:

```
10
50
```

Why?

Because:

```
first = 10
    →
copy 10
    →
second = 10
```

Then:

```
second = 50
```

changes only `second`.

Value Type Copy Flow

```
first
10
    →
Assignment
    →
Copy value
    →
second
10
```

Then:

```
second
10
    →
change
    →
50
```

`first` remains:

```
10
```

Common Value Types

Examples include:

```
byte
sbyte
short
ushort
int
uint
long
ulong

float
double
decimal

bool
char

DateTime
DateOnly
TimeOnly
TimeSpan

Guid

struct types

record struct types

enum types
```

Numeric types, Boolean values, enums, and user-defined structures are value types.

Fundraising Value Type Examples

```
long giftId = 10001;

decimal totalAmount = 2500m;

bool isAnonymous = false;

DateOnly giftDate =
    new DateOnly(
        2026,
        10,
        29);

Guid outboxMessageId =
    Guid.NewGuid();
```

All these variables contain value-type values.

What Is a Reference Type?

Definition

A **reference type variable stores a reference to an object rather than containing the complete object value directly**.

Example:

```
Gift gift =
    new Gift();
```

Conceptually:

```
gift variable
    →
reference
    →
Gift object
```

The object exists separately from the variable holding the reference to it.

Microsoft describes reference-type variables as storing references to their data, which allows multiple variables to refer to the same object.

Reference Type Example

Suppose:

```
public class Gift
{
    public decimal TotalAmount { get; set; }
}
```

Create object:

```
Gift first =
    new Gift
    {
        TotalAmount = 1000m
    };
```

Conceptually:

```
first
    →
Reference A
    →
Gift Object
       TotalAmount = 1000
```

Now:

```
Gift second = first;
```

C# does **not** create a new `Gift` object.

It copies the reference.

Now:

```
first  ──────┐
             ↓
          Gift Object
             ↑
second ──────┘
```

Both variables refer to the same object.

Modifying Through One Reference

```
Gift first =
    new Gift
    {
        TotalAmount = 1000m
    };

Gift second = first;

second.TotalAmount = 5000m;

Console.WriteLine(
    first.TotalAmount);
```

Output:

```
5000
```

Why?

You did not modify:

```
first variable
```

directly.

You modified:

```
the object
```

that both references point to.

Reference Copy Flow

Initially:

```
first
    →
Gift Object
TotalAmount = 1000
```

Assignment:

```
Gift second = first;
```

copies the reference:

```
first  ──┐
         ↓
       Gift Object
         ↑
second ──┘
```

Then:

```
second.TotalAmount = 5000m;
```

changes:

```
Gift Object
```

not merely `second`.

Therefore:

```
first.TotalAmount
```

also returns:

```
5000
```

Common Reference Types

Examples include:

```
class types

record class types

string

object

dynamic

arrays

interface references

delegate types
```

Microsoft specifically identifies classes, interfaces, arrays, delegates, `dynamic`, `object`, and `string` as reference types.

Fundraising Reference Type Examples

```
Gift gift =
    new Gift();

Campaign campaign =
    new Campaign();

string donorName =
    "John Smith";

decimal[] giftAmounts =
{
    100m,
    200m,
    300m
};

List<Gift> gifts =
    new List<Gift>();
```

All of these variables contain references.

Value Type vs Reference Type — Basic Comparison

Value Type	Reference Type
Variable contains value	Variable contains reference
Assignment normally copies value	Assignment copies reference
Copies usually become independent values	Copies can point to same object
Usually cannot be `null` unless nullable	Can generally represent `null`
Examples: `int`, `decimal`, `bool`, structs	Examples: classes, arrays, strings

The word:

```
normally
```

is important because features such as:

```
ref
out
in
```

change how variables are passed or referenced.

Value Semantics

The term **value semantics** generally means that copying a value produces another independent value.

Example:

```
decimal amount1 = 100m;
decimal amount2 = amount1;

amount2 = 500m;
```

Afterward:

```
amount1 = 100

amount2 = 500
```

No shared mutable object state exists between those variables.

Reference Semantics

With normal mutable reference objects:

```
Gift gift1 =
    new Gift
    {
        TotalAmount = 100m
    };

Gift gift2 = gift1;
```

Both references identify the same object.

Changing:

```
gift2.TotalAmount = 500m;
```

changes the shared object's property.

So:

```
gift1.TotalAmount = 500
gift2.TotalAmount = 500
```

Important Distinction: Variable vs Object

Beginners often think:

```
gift variable = Gift object
```

A more accurate model is:

```
gift variable
    →
contains reference
    →
Gift object
```

The variable and the object are different things.

This becomes very important when:

```
reassigning variables

passing objects to methods

comparing references

working with null
```

Reassigning a Reference Variable

Suppose:

```
Gift first =
    new Gift
    {
        TotalAmount = 1000m
    };

Gift second = first;
```

Initially:

```
first  ──┐
         ↓
      Gift Object A
         ↑
second ──┘
```

Then:

```
second =
    new Gift
    {
        TotalAmount = 9000m
    };
```

Now:

```
first
    →
Gift Object A
TotalAmount = 1000

second
    →
Gift Object B
TotalAmount = 9000
```

Reassigning `second` did **not** change `first`.

Object Mutation vs Reference Reassignment

This distinction is essential.

Object Mutation

```
second.TotalAmount = 5000m;
```

means:

```
Change object state
```

If `first` and `second` refer to the same object, both observe the new state.

Reference Reassignment

```
second = new Gift();
```

means:

```
Change which object second points to
```

It does not automatically change:

```
first
```

Reference Reassignment Flow

Before:

```
first  ──┐
         ↓
     Object A
         ↑
second ──┘
```

After:

```
second = new Gift();
```

```
first
    →
Object A


second
    →
Object B
```

struct Is a Value Type

A `struct` creates a user-defined value type.

Example:

```
public struct GiftSummary
{
    public int Count { get; set; }

    public decimal Total { get; set; }
}
```

Usage:

```
GiftSummary first =
    new GiftSummary
    {
        Count = 5,
        Total = 1000m
    };

GiftSummary second = first;
```

The entire struct value is copied.

Struct Copy Example

```
GiftSummary first =
    new GiftSummary
    {
        Count = 5,
        Total = 1000m
    };

GiftSummary second = first;

second.Total = 5000m;
```

Now:

```
first.Total  = 1000
second.Total = 5000
```

because they are separate struct values.

Struct Copy Flow

```
first
    →
Count = 5
Total = 1000

Assignment
    →
copy fields

second
    →
Count = 5
Total = 1000
```

Then changing `second` does not automatically affect `first`.

But Structs Can Contain References

This is where things become more advanced.

Consider:

```
public struct GiftBatch
{
    public List<long> GiftIds { get; set; }
}
```

Now:

```
GiftBatch first =
    new GiftBatch
    {
        GiftIds =
            new List<long>
            {
                1,
                2
            }
    };

GiftBatch second = first;
```

The struct itself is copied.

But the field:

```
GiftIds
```

is a reference.

So the reference value is copied.

Both structs can now refer to the same `List<long>`.

Struct Containing Reference Flow

Before copy:

```
first struct
    →
GiftIds reference
    →
List Object
[1, 2]
```

After:

```
GiftBatch second = first;
```

```
first.GiftIds  ──┐
                 ↓
               List Object
                 ↑
second.GiftIds ──┘
```

Then:

```
second.GiftIds.Add(
    3);
```

also affects what is visible through:

```
first.GiftIds
```

because the nested object is shared.

Important Rule

Value-type copying is:

```
field-by-field value copying
```

It is **not necessarily deep cloning**.

If a struct contains reference-type fields:

```
the references are copied
```

not the referenced objects.

---

enum Is a Value Type

Example:

```
public enum GiftStatus
{
    Pending,
    Processed,
    Refunded
}
```

Usage:

```
GiftStatus status =
    GiftStatus.Pending;
```

`GiftStatus` is a value type.

When assigned:

```
GiftStatus another =
    status;
```

its value is copied.

Records and Type Category

C# has both:

```
record class

record struct
```

A:

```
record class
```

is a reference type.

A:

```
record struct
```

is a value type.

Example:

```
public record GiftDto(
    long GiftId,
    decimal Amount);
```

Without `struct`, this is a record class and therefore a reference type.

Example:

```
public readonly record struct Money(
    decimal Amount,
    string Currency);
```

is a value type.

We will study records in depth later.

Arrays Are Reference Types

This is important because arrays contain values but the array itself is a reference-type object.

Example:

```
int[] first =
{
    10,
    20,
    30
};

int[] second = first;

second[0] = 999;
```

Now:

```
Console.WriteLine(
    first[0]);
```

prints:

```
999
```

Why?

Both variables reference the same array object.

Array Reference Flow

```
first  ──┐
         ↓
      [10,20,30]
         ↑
second ──┘
```

Change through:

```
second[0] = 999;
```

shared object becomes:

```
[999,20,30]
```

Copying an Array Reference Is Not Copying Array Elements

This:

```
int[] second = first;
```

does **not** duplicate the array.

If you need a separate array, use an actual copy operation.

For example:

```
int[] second =
    (int[])first.Clone();
```

or appropriate collection copying.

Then:

```
first
    →
Array A

second
    →
Array B
```

Lists Are Reference Types

`List<T>` is a class.

Therefore:

```
List<long> first =
    new List<long>
    {
        1,
        2
    };

List<long> second =
    first;
```

means:

```
two references
    →
one List object
```

Then:

```
second.Add(
    3);
```

also changes what you see through `first`.

string Is a Reference Type

This surprises many beginners.

```
string name =
    "John";
```

`string` is a reference type.

But strings behave differently from many mutable classes because:

```
strings are immutable
```

Once created, a string object's textual content does not change.

String Example

```
string first =
    "Annual";

string second =
    first;
```

Both may refer to the same string value.

Then:

```
second +=
    " Campaign";
```

You might think the original string was modified.

It was not.

A new string is created.

So:

```
first  = "Annual"

second = "Annual Campaign"
```

String Immutability Flow

Before:

```
first  ──┐
         ↓
      "Annual"
         ↑
second ──┘
```

After:

```
second += " Campaign";
```

```
first
    →
"Annual"

second
    →
"Annual Campaign"
```

This is why string can sometimes feel value-like even though it is a reference type.

We will study string immutability deeply in the dedicated Strings topic.

object Is a Reference Type

`object` is:

```
System.Object
```

and is the root type of C#'s unified type system.

Every C# type can ultimately be treated as an `object`.

Reference types can be viewed as `object` references.

Value types can be converted to `object` using:

```
boxing
```

Microsoft's language specification describes C# as a unified type system in which every type ultimately relates to `object`; value types participate through boxing and unboxing.

Interfaces Are Reference Types

Example:

```
public interface IGiftProcessor
{
    void Process();
}
```

A variable:

```
IGiftProcessor processor;
```

is a reference-type variable.

It may refer to an object whose concrete class implements the interface.

Example:

```
IGiftProcessor processor =
    new CreditCardGiftProcessor();
```

Flow:

```
interface variable
    →
reference
    →
concrete object
```

Value Types Can Implement Interfaces

This is important for boxing.

For example:

```
int value = 10;
```

`int` implements interfaces such as:

```
IComparable<int>
```

and others.

If a value type is converted to a non-generic interface reference:

```
IComparable comparable =
    value;
```

boxing can occur.

We will return to this under boxing.

Delegates Are Reference Types

A delegate represents references to callable methods.

Example later:

```
Action action =
    ProcessGift;
```

`Action` is a delegate type, therefore a reference type.

We will study delegates in detail later.

Value Type Assignment

Let's examine copying carefully.

```
decimal amount1 = 500m;
decimal amount2 = amount1;
```

Conceptually:

```
amount1
500

amount2
500
```

Then:

```
amount2 += 100m;
```

results:

```
amount1 = 500

amount2 = 600
```

Reference Type Assignment

```
Gift gift1 =
    new Gift
    {
        TotalAmount = 500m
    };

Gift gift2 =
    gift1;
```

Conceptually:

```
gift1 ──┐
        ↓
     Gift Object
     Amount = 500
        ↑
gift2 ──┘
```

Then:

```
gift2.TotalAmount += 100m;
```

object becomes:

```
Amount = 600
```

Both references observe:

```
600
```

Copying a Class Reference Does Not Clone the Object

This is an important interview statement.

This:

```
Gift copy =
    original;
```

does **not** create a separate `Gift`.

It creates another reference to the same object.

If you need independent object state, you need an actual cloning/copying strategy.

Examples could include:

```
copy constructor

factory method

record with-expression

manual mapping

serialization-based copying
```

depending on the type and requirements.

Shallow Copy vs Deep Copy

This is an advanced but useful concept.

Shallow Copy

Copies the outer object's fields.

Reference fields still point to the same nested objects.

Example concept:

```
Gift
    →
Donor reference
```

A shallow copy may produce:

```
Gift A ──→ Donor Object
Gift B ──→ same Donor Object
```

Deep Copy

A deep copy creates independent nested objects as required.

Conceptually:

```
Gift A
    →
Donor A

Gift B
    →
Donor B
```

The correct strategy depends on the domain.

Do not automatically deep clone everything.

Value Type Parameters — Review

You learned method parameter passing in the previous topic.

Example:

```
static void ChangeAmount(
    decimal amount)
{
    amount = 5000m;
}
```

Call:

```
decimal giftAmount =
    1000m;

ChangeAmount(
    giftAmount);
```

The method receives a copy.

Therefore:

```
giftAmount remains 1000
```

Reference Type Parameters — Review

```
static void ChangeGift(
    Gift gift)
{
    gift.TotalAmount =
        5000m;
}
```

Call:

```
Gift gift =
    new Gift
    {
        TotalAmount = 1000m
    };

ChangeGift(
    gift);
```

The reference is copied.

Both references identify the same object.

Therefore the object can be mutated.

Reference Parameter Reassignment

```
static void ReplaceGift(
    Gift gift)
{
    gift =
        new Gift
        {
            TotalAmount =
                9000m
        };
}
```

Caller:

```
Gift gift =
    new Gift
    {
        TotalAmount =
            1000m
    };

ReplaceGift(
    gift);
```

Caller still references original object.

Why?

```
Caller reference
    →
copied

Method reassigns
its copy only
```

---

ref Changes the Variable Relationship

If you use:

```
static void ReplaceGift(
    ref Gift gift)
```

then the method receives an alias to the caller's variable.

Now it can replace what the caller variable references.

Example:

```
static void ReplaceGift(
    ref Gift gift)
{
    gift =
        new Gift
        {
            TotalAmount =
                9000m
        };
}
```

Call:

```
Gift gift =
    new Gift
    {
        TotalAmount =
            1000m
    };

ReplaceGift(
    ref gift);
```

Now:

```
gift.TotalAmount = 9000
```

because the actual caller variable was reassigned.

Stack and Heap — Important Warning

A common beginner explanation says:

```
Value types go on stack.

Reference types go on heap.
```

This is **too simplistic and not universally correct**.

Do not use that as an absolute rule in interviews.

The more accurate model is:

```
Value type
    →
describes value semantics

Reference type
    →
describes reference semantics
```

Storage location depends on context and runtime implementation.

What Is the Stack?

At a high level, each executing thread uses a stack containing method-call information.

A method call may have a stack frame containing information such as:

```
Parameters

Local variables

Return information

Temporary execution state
```

Conceptually:

```
Main()
    →
ProcessGift()
        →
CalculateFee()
```

can produce nested call frames.

Stack Frame Flow

```
Main frame
    →
ProcessGift frame
        →
CalculateFee frame
```

When:

```
CalculateFee
```

returns:

```
its frame is removed
```

Then execution continues in:

```
ProcessGift
```

What Is the Managed Heap?

Objects used by managed .NET applications are commonly allocated in managed memory controlled by the garbage collector.

Conceptually:

```
new Gift()
    →
managed object allocation
    →
CLR / GC-managed memory
```

The garbage collector eventually reclaims unreachable managed objects.

We will study:

```
GC generations
Gen 0
Gen 1
Gen 2
LOH
GC roots
allocations
```

in the dedicated CLR and memory-management topic.

Why “Value Type = Stack” Is Wrong

Consider a class:

```
public class Gift
{
    public decimal TotalAmount { get; set; }
}
```

`decimal` is a value type.

But `TotalAmount` is stored as part of the `Gift` object's state.

If the `Gift` object exists in managed heap storage, the `decimal` value is stored **inside that object**.

Therefore:

```
decimal
```

does not mean:

```
must live on stack
```

Value Type Inside an Object

Conceptually:

```
Managed Gift Object
    →
GiftId        : long
TotalAmount   : decimal
IsAnonymous   : bool
```

These value-type fields exist as part of the object's storage.

They are not separate boxed objects merely because they are value types.

Value Types Inside Arrays

Consider:

```
int[] values =
{
    10,
    20,
    30
};
```

The array is a reference-type object.

The `int` values are stored inline as elements of the array.

Conceptually:

```
Array Object
    →
[10][20][30]
```

Again:

```
value type
```

does not mean:

```
always on stack
```

Reference Variables Can Also Exist in Objects

Consider:

```
public class Gift
{
    public Campaign? Campaign { get; set; }
}
```

The `Gift` object contains a reference field.

So a reference value itself can exist inside another heap object.

Therefore:

```
reference variable = stack
```

is also not a universal rule.

Better Stack/Heap Explanation

Use this interview explanation:

> Value type versus reference type primarily describes **semantics**, not a guaranteed storage location. Value-type variables contain their values; reference-type variables contain references to objects. Local variables may participate in stack-frame storage, while objects are generally managed by the runtime in managed memory, and value types can also be embedded inside objects or arrays. Runtime/JIT optimizations can further affect physical storage.

That is much more technically accurate.

Runtime Optimizations

The CLR/JIT may optimize storage.

For example, advanced runtime optimizations can sometimes avoid or transform allocations depending on the code and runtime implementation.

Therefore application developers should usually reason about:

```
semantic behavior
```

rather than trying to predict exact physical addresses.

When performance matters:

```
measure
profile
benchmark
```

instead of relying on oversimplified memory folklore.

What Is Boxing?

Definition

Boxing converts a value type into an `object` reference or an interface reference implemented by that value type.

Example:

```
int number = 123;

object boxed =
    number;
```

The conversion:

```
int
    →
object
```

requires boxing.

Microsoft defines boxing exactly this way and explains that the CLR creates an object instance containing a copy of the value.

Boxing Flow

Start:

```
number
    →
int value
123
```

Then:

```
object boxed =
    number;
```

Conceptually:

```
int value
123
    →
Boxing
    →
new managed object
containing copy 123
    →
boxed reference
points to object
```

Boxing Creates a Copy

This is very important.

```
int number = 10;

object boxed = number;
```

The boxed object contains a copy:

```
10
```

Now:

```
number = 50;
```

does not change the boxed copy.

So:

```
Console.WriteLine(
    boxed);
```

still represents:

```
10
```

Boxing Copy Flow

Before boxing:

```
number = 10
```

Box:

```
number 10
    →
copy
    →
boxed object 10
```

Then:

```
number = 50
```

Final:

```
number = 50

boxed object = 10
```

Why Boxing Exists

C# has a unified type system.

We can write:

```
object value =
    10;
```

even though:

```
10
```

is an `int` value type.

Boxing provides the bridge between:

```
value types
```

and:

```
reference-type object model
```

Microsoft explicitly describes boxing/unboxing as enabling C#'s unified view in which values of any type can ultimately be treated as `object`.

Boxing Is Usually Implicit

You normally do not write:

```
Box(...)
```

The conversion happens automatically.

Example:

```
int number = 100;

object value =
    number;
```

This performs implicit boxing.

More Boxing Examples

```
decimal amount =
    1000m;

object value =
    amount;
```

Boxes the `decimal`.

```
bool active =
    true;

object value =
    active;
```

Boxes the `bool`.

```
DateTime now =
    DateTime.UtcNow;

object value =
    now;
```

Boxes the `DateTime`.

Boxing to an Interface

Boxing is not limited to `object`.

A value type converted to an interface it implements may also need boxing.

Example:

```
int number = 10;

IComparable comparable =
    number;
```

Conceptually:

```
int value
    →
boxing
    →
object implementing IComparable
    →
interface reference
```

Microsoft's boxing documentation defines boxing as conversion of a value type to `object` **or to an interface type implemented by the value type**.

Why Interface Boxing Matters

This can become important in high-performance generic code.

Example:

```
struct
    →
converted to interface
    →
boxing allocation
```

Repeated millions of times, that may create significant garbage-collector pressure.

For normal business code, this is usually not the first performance concern.

But it is important advanced knowledge.

What Is Unboxing?

Definition

Unboxing extracts a value type from a previously boxed object.

Example:

```
int number = 123;

object boxed =
    number;

int unboxed =
    (int)boxed;
```

Flow:

```
int 123
    →
box
    →
object containing 123
    →
unbox
    →
int 123
```

Microsoft documents unboxing as an explicit operation that verifies the boxed value has the expected value type and then copies that value out.

Unboxing Is Explicit

Boxing:

```
object boxed =
    number;
```

is implicit.

Unboxing:

```
int value =
    (int)boxed;
```

requires an explicit cast.

Why?

Because an `object` could contain many possible runtime types.

The compiler cannot assume that it contains an `int`.

Unboxing Runtime Check

When you write:

```
int result =
    (int)boxed;
```

runtime conceptually does:

```
boxed reference null?
    →
wrong

Does object contain boxed int?
    →
Yes
    →
copy int value out

No
    →
InvalidCastException
```

Exact Type Requirement

This is a very important interview point.

Suppose:

```
object value =
    100;
```

The boxed type is:

```
int
```

This works:

```
int number =
    (int)value;
```

This does **not** work:

```
long number =
    (long)value;
```

even though normal numeric conversion exists from:

```
int → long
```

Why?

Because unboxing requires the boxed value to match the appropriate value type.

Microsoft specifically notes that unboxing must target the type that was boxed; incompatible value types produce `InvalidCastException`.

Correct int-to-long After Boxing

If:

```
object value =
    100;
```

and you need a `long`:

```
long number =
    (long)(int)value;
```

Flow:

```
boxed int
    →
unbox to int
    →
numeric conversion
    →
long
```

These are two different operations.

Invalid Unboxing Example

```
object value =
    100;

short number =
    (short)value;
```

This throws:

```
InvalidCastException
```

because the boxed object contains:

```
System.Int32
```

not:

```
System.Int16
```

This is a classic C# interview trap.

Unboxing null

Example:

```
object? value =
    null;

int number =
    (int)value;
```

This cannot produce a valid `int`.

Unboxing a null object reference to a non-nullable value type results in an exception. Microsoft documents this as a `NullReferenceException`.

Safer Type Checking

Instead of blindly unboxing:

```
int number =
    (int)value;
```

when runtime type is uncertain, use pattern matching:

```
if (value is int number)
{
    Console.WriteLine(
        number);
}
```

Flow:

```
value
    →
is boxed int?
    →
Yes
    →
extract int safely

No
    →
skip branch
```

This avoids an invalid cast when the object contains another type.

Boxing and object Collections

Before generic collections became standard, code often used collections such as:

```
ArrayList
```

which stores elements as:

```
object
```

Example:

```
var values =
    new System.Collections.ArrayList();

values.Add(
    10);

values.Add(
    20);
```

Each `int` must be represented as an `object`.

That can require boxing.

Generic Collections Avoid Much Boxing

Modern code usually uses:

```
List<int>
```

Example:

```
var values =
    new List<int>
    {
        10,
        20
    };
```

The generic collection is strongly typed for:

```
int
```

and does not need to treat every element as an `object`.

This is one major advantage of generics.

Old Collection vs Generic Collection

```
ArrayList
    →
stores object
    →
int may box


List<int>
    →
stores int elements
    →
no object boxing simply for storage
```

Later in the Generics and Collections topics we will study this deeply.

Boxing with object[]

Consider:

```
object[] values =
{
    10,
    20m,
    true
};
```

Each value type used as an element must be converted to:

```
object
```

so boxing occurs for:

```
10

20m

true
```

The array itself is also a reference-type object.

Boxing Can Be Hidden

You might not explicitly write:

```
object value =
    number;
```

but boxing can still occur when an API expects `object`.

For example:

```
void Log(
    object value)
{
}
```

Call:

```
int giftCount =
    10;

Log(
    giftCount);
```

The `int` must be represented as an `object`.

That can box it.

Boxing Through Interfaces

Suppose:

```
void Compare(
    IComparable value)
{
}
```

Call:

```
int number = 10;

Compare(
    number);
```

Because `int` is a value type being converted to a non-generic interface reference:

```
boxing may occur
```

Generics Help Preserve Value Types

Consider:

```
static void Print<T>(
    T value)
{
    Console.WriteLine(
        value);
}
```

Call:

```
Print(
    10);
```

Here `T` becomes:

```
int
```

Generics allow code to preserve type information instead of forcing everything through `object`.

This often reduces unnecessary boxing.

We will study this carefully in the Generics topic.

Boxing Performance

Boxing is more expensive than simple value assignment.

Why?

Because boxing generally involves:

```
Create managed object
    →
Copy value into object
    →
Return/store reference
```

The resulting managed allocation eventually contributes to garbage-collection work.

Microsoft explicitly notes that boxing and unboxing are more computationally expensive than simple assignments because boxing creates an object allocation, while unboxing also requires type checking and value extraction.

Boxing Cost Flow

```
Value Type
    →
Boxing
    →
Managed Allocation
    →
Value Copy
    →
Reference
    →
Later GC responsibility
```

One boxing operation is rarely a meaningful problem.

Millions of repeated boxing operations inside hot code can matter.

Example of Repeated Boxing

```
object value;

for (int i = 0;
     i < 1_000_000;
     i++)
{
    value = i;
}
```

Each assignment can involve boxing a new `int`.

Conceptually:

```
1,000,000 iterations
    →
many boxed objects
    →
more allocations
    →
more GC pressure
```

This is not how you should normally design numeric processing.

Better Generic/Typed Approach

```
int value = 0;

for (int i = 0;
     i < 1_000_000;
     i++)
{
    value = i;
}
```

No object boxing is needed merely to store the integer.

Do Not Fear Boxing Everywhere

Avoid the opposite mistake:

> Boxing exists, therefore every use of `object` is bad.

No.

Use `object` when the abstraction genuinely requires it.

Examples include:

```
some reflection APIs

framework infrastructure

general metadata

interop

heterogeneous values
```

Performance optimization should be based on:

```
measurement
```

not fear of individual allocations.

Nullable Value Types — Boxing Behavior

This is advanced but useful.

Suppose:

```
int? value =
    10;
```

When boxing a nullable value type:

- if it contains a value, the underlying value type is boxed;
- if it has no value, the result is a null reference.

Microsoft documents this behavior explicitly for `Nullable<T>`.

Nullable Boxing with Value

```
int? number =
    10;

object? boxed =
    number;
```

The boxed object contains:

```
boxed int 10
```

not a boxed:

```
Nullable<int>
```

object in the ordinary sense.

Nullable Boxing with null

```
int? number =
    null;

object? boxed =
    number;
```

Result:

```
boxed == null
```

No boxed integer object exists.

Nullable Unboxing

A boxed underlying value can be unboxed into the corresponding nullable value type.

Example concept:

```
object boxed =
    10;

int? number =
    (int?)boxed;
```

Microsoft documents this as valid behavior.

We will study nullable values more deeply in Topic 7.

Equality — Value Types

For many value types:

```
int first = 10;
int second = 10;

bool equal =
    first == second;
```

compares values.

Result:

```
true
```

Struct Equality

User-defined structs inherit value-oriented behavior from `System.ValueType`, but exact equality support depends on the type and operators it defines.

For example, a normal user-defined struct does not automatically gain every possible `==` operator form.

You can use:

```
first.Equals(
    second);
```

or define appropriate equality behavior.

Modern record structs generate value-based equality members automatically.

We will study equality much more deeply in OOP and Records.

Equality — Reference Types

For normal classes, there are multiple equality concepts:

```
Reference identity

Value equality
```

Consider:

```
Gift first =
    new Gift
    {
        TotalAmount = 1000m
    };

Gift second =
    new Gift
    {
        TotalAmount = 1000m
    };
```

They contain similar data.

But they are two different objects.

Conceptually:

```
first
    →
Object A

second
    →
Object B
```

Reference identity is different.

ReferenceEquals

You can explicitly test whether two variables refer to the exact same object:

```
bool sameObject =
    ReferenceEquals(
        first,
        second);
```

If they refer to different objects:

```
false
```

If:

```
Gift second =
    first;
```

then:

```
ReferenceEquals(
    first,
    second)
```

returns:

```
true
```

== for Classes

For a normal class that does not overload `==`, the operator typically represents reference identity behavior inherited from reference semantics.

But some reference types define their own equality operators.

Important examples:

```
string

record class

many framework types
```

So do not blindly assume:

```
== always means reference comparison for all reference types
```

Strings Use Value Equality

```
string first =
    "Gift";

string second =
    "Gift";

bool equal =
    first == second;
```

Result:

```
true
```

because `string` overloads equality to compare textual content.

This is another reason `string` feels different from many mutable classes even though it is a reference type.

Records Use Value-Oriented Equality

A record class is a reference type, but records generate value-based equality semantics based on their components.

Example:

```
public record GiftDto(
    long GiftId,
    decimal Amount);
```

Two separate record objects with equivalent values can compare as equal according to record equality semantics.

We will cover this fully in the Records topic.

Value Type Does Not Mean Immutable

This is another common misconception.

Value types can be mutable.

Example:

```
public struct GiftSummary
{
    public decimal Total { get; set; }
}
```

This struct can change:

```
GiftSummary summary =
    new GiftSummary();

summary.Total =
    1000m;
```

So:

```
value type
```

does not automatically mean:

```
immutable
```

Mutable Struct Problems

Mutable structs can create confusing behavior because value copying happens automatically.

Example:

```
GiftSummary first =
    new GiftSummary
    {
        Total = 1000m
    };

GiftSummary second =
    first;

second.Total =
    5000m;
```

Now:

```
first.Total  = 1000
second.Total = 5000
```

A developer expecting shared object-like behavior may be surprised.

Why Immutable Structs Are Often Preferred

Small immutable value types are usually easier to reason about.

Example:

```
public readonly struct Money
{
    public Money(
        decimal amount)
    {
        Amount =
            amount;
    }

    public decimal Amount
    {
        get;
    }
}
```

Once created:

```
Money value
```

cannot be modified through its public API.

A changed amount requires a new value.

readonly struct

A:

```
readonly struct
```

expresses that instance fields should not be mutated after construction.

Example:

```
public readonly struct Money
{
    public Money(
        decimal amount,
        string currency)
    {
        Amount =
            amount;

        Currency =
            currency;
    }

    public decimal Amount
    {
        get;
    }

    public string Currency
    {
        get;
    }
}
```

This can be a good pattern for value objects when the type is small and genuinely has value semantics.

Class vs Struct — Beginner Rule

Do not choose `struct` simply because:

```
it might be faster
```

Use a struct when the concept genuinely behaves like a small value.

Examples:

```
Point

Date

Time interval

small measurement

small immutable identifier/value object
```

Use a class when the concept has:

```
identity

shared mutable state

complex lifecycle

large object state

inheritance requirements
```

Fundraising Example — Gift Should Usually Be a Class

A `Gift` is a business entity with identity:

```
GiftId
```

Two gifts with the same amount are not necessarily the same gift.

Therefore:

```
public class Gift
```

is usually natural.

Why?

```
Gift has identity

Gift changes over time

Gift participates in relationships

Gift has lifecycle
```

Fundraising Example — Money Could Be a Value Object

A value such as:

```
100 USD
```

can be modeled as a value.

Example:

```
public readonly record struct Money(
    decimal Amount,
    string Currency);
```

Conceptually:

```
100 USD
```

is defined by its values rather than by an independent database identity.

This can make a value type appropriate.

Whether you actually implement `Money` this way depends on application requirements.

Entity vs Value Concept

Entity:

```
Gift #1001
```

identity matters.

Value concept:

```
$100 USD
```

value itself matters.

Simplified:

```
Entity
    →
usually class/reference semantics

Value Object
    →
may be class or struct depending design
```

DDD value objects do not *have* to be C# structs.

Do not confuse:

```
domain value object
```

with:

```
CLR value type
```

They are related concepts but not identical.

Large Structs and Copying

Because structs are copied by value, large structs can become expensive when repeatedly:

```
assigned

passed to methods

returned

stored/moved
```

Example concept:

```
struct containing
20 fields
```

may require copying those fields repeatedly.

This is one reason structs are normally kept relatively small.

in Parameter for Large Structs

You learned:

```
in
```

in the previous topic.

Example:

```
static decimal Calculate(
    in LargeGiftStatistics value)
```

can allow a large struct to be passed by read-only reference instead of copying the entire value.

But do not apply this blindly.

Use it when:

```
the struct is large
+
performance matters
+
measurement/design supports it
```

ref Returns and ref Locals — Preview

Advanced C# can allow references to variables rather than copies.

Example concepts:

```
ref local

ref return

ref readonly
```

These features let code work with an existing storage location.

They are useful in:

```
performance-sensitive APIs

Span<T>

low-level libraries

large structs
```

We will revisit reference semantics in advanced C#.

Value Type in List

```
var amounts =
    new List<decimal>();
```

The generic collection is specialized for:

```
decimal
```

It can store values without requiring ordinary object boxing just to represent each element.

This is one reason modern generics are important.

Struct in List

Suppose:

```
public struct GiftSummary
{
    public decimal Total { get; set; }
}
```

Then:

```
List<GiftSummary> summaries =
    new();
```

elements are struct values.

Retrieving one normally gives a value copy.

This is another reason mutable structs can be confusing in collections.

Mutable Struct in Collection Problem

Imagine:

```
GiftSummary summary =
    summaries[0];

summary.Total =
    5000m;
```

You may only be changing the local copy.

The collection element may remain unchanged unless reassigned appropriately.

This is one of the practical reasons immutable structs are preferred.

Reference Type in List

```
List<Gift> gifts =
    new();
```

The list stores references to `Gift` objects.

If you retrieve:

```
Gift gift =
    gifts[0];
```

and then mutate:

```
gift.TotalAmount =
    5000m;
```

you modify the same object referenced by the list entry.

Collection Comparison

```
List<GiftSummary>
    →
struct values

List<Gift>
    →
object references
```

That difference influences mutation behavior.

Class Field Containing a Struct

Example:

```
public class CampaignStatistics
{
    public GiftSummary Summary
    {
        get;
        set;
    }
}
```

`CampaignStatistics` is a reference object.

`Summary` is a value stored within the object's state.

This shows again:

```
value type
```

does not imply:

```
stack only
```

Struct Containing a Class Reference

Example:

```
public struct GiftInfo
{
    public Constituent? Constituent
    {
        get;
        set;
    }
}
```

`GiftInfo` is a value type.

But its field/property can hold:

```
reference
```

to another object.

When the struct is copied:

```
reference is copied
```

but the `Constituent` object is not cloned.

Reference Cycles

Reference objects can point to each other.

Example:

```
Constituent
    →
Gifts

Gift
    →
Constituent
```

This can form an object graph.

The .NET garbage collector does not rely only on simple reference counting, so cycles by themselves do not permanently prevent collection if the entire graph becomes unreachable.

We will explore this in GC/memory management.

Null and Reference Types

A reference variable can represent:

```
no object
```

using:

```
null
```

Example:

```
Gift? gift =
    null;
```

Conceptually:

```
gift
    →
no object reference
```

Value types normally cannot represent `null` unless wrapped as:

```
Nullable<T>
```

such as:

```
long? campaignId =
    null;
```

The next topic covers nullability in depth.

Value Type Default

Example:

```
int value =
    default;
```

Result:

```
0
```

For:

```
bool value =
    default;
```

result:

```
false
```

For a struct, its fields receive their default values.

Reference Type Default

For a reference type:

```
Gift? gift =
    default;
```

the default reference is:

```
null
```

Nullable reference type annotations affect compiler analysis but do not change the underlying fact that references can represent null at runtime.

We will cover this next.

Copy Semantics in Fundraising Platform

Consider:

```
public class Gift
{
    public long GiftId { get; set; }

    public decimal TotalAmount { get; set; }
}
```

Code:

```
Gift gift1 =
    new Gift
    {
        GiftId = 1001,
        TotalAmount = 1000m
    };

Gift gift2 =
    gift1;

gift2.TotalAmount =
    1500m;
```

Result:

```
gift1.TotalAmount = 1500
gift2.TotalAmount = 1500
```

because:

```
one Gift object
+
two references
```

Compare with Struct Fundraising Value

```
public readonly record struct Money(
    decimal Amount,
    string Currency);
```

Code:

```
Money first =
    new Money(
        1000m,
        "USD");

Money second =
    first;
```

Now two independent value copies exist.

If you create:

```
second =
    new Money(
        1500m,
        "USD");
```

`first` remains:

```
1000 USD
```

Reference Identity in Fundraising

Suppose:

```
Gift gift1 =
    LoadGift();

Gift gift2 =
    gift1;
```

Then:

```
ReferenceEquals(
    gift1,
    gift2)
```

is:

```
true
```

because both references point to one object.

Separate Objects with Same Data

```
Gift gift1 =
    new Gift
    {
        GiftId = 1001,
        TotalAmount = 1000m
    };

Gift gift2 =
    new Gift
    {
        GiftId = 1001,
        TotalAmount = 1000m
    };
```

Even though their property values are the same:

```
they are separate objects
```

unless equality behavior is explicitly defined to treat them otherwise.

Why This Matters in EF Core Later

Entity Framework Core works heavily with object identity and tracking.

For example:

```
DbContext
    →
tracks entity object instances
```

Understanding:

```
same object
vs
different object with same key
```

becomes important for:

```
change tracking
identity resolution
entity states
```

Later EF Core topics will build directly on today's reference-semantics foundation.

Why This Matters in Dependency Injection

Suppose a DI container gives multiple consumers the same singleton object.

Conceptually:

```
Service A ──┐
            ↓
        Singleton Object
            ↑
Service B ──┘
```

Both hold references to shared state.

If the object is mutable:

```
one consumer can affect another
```

That is one reason singleton services require careful thread-safety design.

Later DI topics depend on this reference-type behavior.

Why This Matters in Async and Concurrency

If several tasks hold references to one mutable object:

```
Task A ──┐
         ↓
      Shared Object
         ↑
Task B ──┘
```

both may modify it concurrently.

This can create:

```
race conditions
```

Value copying and immutability can reduce some shared-state risks.

Later concurrency topics will build on this.

Why This Matters in ASP.NET Core

Suppose you store a mutable object as a singleton:

```
all requests
    →
same service object
```

Because it is shared by reference across requests, unsafe mutation may affect multiple users simultaneously.

Understanding reference types therefore directly affects:

```
service lifetimes

thread safety

caching

request state
```

Common Beginner Mistake — “Classes Are Passed by Reference”

This statement is incomplete.

More accurate:

> A class variable contains a reference, and normal parameter passing passes **a copy of that reference by value**.

Example:

```
void Process(
    Gift gift)
```

does not mean:

```
ref Gift gift
```

These are different.

Common Beginner Mistake — “Value Types Are Always on Stack”

Incorrect as a universal rule.

A value type can exist:

```
as a local

inside a class object

inside an array

inside another struct

inside runtime-generated storage
```

Reason about:

```
value semantics
```

rather than assuming location.

Common Beginner Mistake — “Reference Types Are Always Heap Variables”

The object is normally managed in managed object storage, but the **reference value itself** can exist in various locations:

```
local variable

object field

array element

closure

other runtime structures
```

Again:

```
type category
≠
simple storage-location rule
```

Common Beginner Mistake — Thinking Assignment Clones an Object

This:

```
Gift second =
    first;
```

does not create:

```
Gift Object B
```

It copies the reference to:

```
Gift Object A
```

Common Beginner Mistake — Thinking Struct Copy Is Deep Copy

If a struct contains:

```
List<T>
```

copying the struct copies:

```
the list reference
```

not:

```
the entire list object
```

Common Beginner Mistake — Wrong Unboxing Type

Bad:

```
object value =
    100;

long number =
    (long)value;
```

The boxed value is an:

```
int
```

Correct:

```
long number =
    (long)(int)value;
```

Common Beginner Mistake — Assuming Casting and Unboxing Are the Same

Example:

```
int number = 10;

long larger =
    number;
```

This is numeric conversion.

But:

```
object boxed =
    number;

int result =
    (int)boxed;
```

is unboxing.

Unboxing is a specific operation involving a boxed value type.

Common Beginner Mistake — Using object When Generic Type Is Available

Avoid:

```
List<object> values =
    new();
```

for a collection that should contain only integers.

Prefer:

```
List<int> values =
    new();
```

Benefits:

```
type safety

clearer code

less casting

avoids boxing for ordinary integer storage
```

Common Beginner Mistake — Mutable Struct for Entity

Avoid modeling something such as:

```
Gift
```

as a large mutable struct merely because:

```
struct sounds faster
```

Business entities usually benefit from reference identity semantics.

Common Beginner Mistake — Using class for Every Tiny Value

The opposite extreme can also be unnecessary.

Small immutable mathematical/value concepts may naturally fit value types.

Examples:

```
coordinates
small measurements
lightweight IDs
```

when semantics justify it.

Choose based on behavior, not fashion.

Performance Considerations

Value Copy Cost

Small values such as:

```
int
long
bool
```

are cheap to copy.

Do not introduce complicated references to avoid copying tiny values.

Large Struct Copy Cost

A large struct may be copied during:

```
assignment

method arguments

returns

collection access
```

That can matter in performance-sensitive code.

This is one reason large data models are normally classes rather than giant structs.

Boxing Allocation Cost

Boxing can involve:

```
managed allocation

copy

later GC work
```

Avoid repeated boxing in hot loops when there is a naturally strongly typed alternative.

Unboxing Cost

Unboxing involves:

```
runtime type validation

value extraction/copy
```

It is less expensive than creating a boxed object but still more work than directly using a strongly typed value.

Microsoft calls both boxing and unboxing computationally more expensive than simple assignment.

Generics and Performance

Prefer:

```
List<int>
```

instead of old object-based containers for integer collections.

Prefer generic abstractions where appropriate:

```
IComparer<T>
```

rather than forcing everything into:

```
object
```

This improves:

```
type safety
+
often performance
```

Avoid Premature Struct Optimization

Do not convert classes into structs simply because:

```
struct avoids allocation
```

That reasoning ignores:

```
copy costs

boxing

semantics

mutation problems

API design

collection behavior
```

Measure real application performance first.

Security and Correctness Considerations

This topic is mostly about semantics and performance, but incorrect reference handling can create correctness/security issues.

For example, suppose a security configuration object is shared:

```
Request A ──┐
            ↓
      Shared Config
            ↑
Request B ──┘
```

If one request mutates state that was intended to be immutable:

```
other requests may observe unintended values
```

This is why shared application configuration should generally be treated carefully and often modeled immutably.

Shared Mutable State

Shared references create shared mutable state.

Conceptually:

```
Reference A
Reference B
Reference C
    →
same mutable object
```

If all can modify it:

```
behavior becomes harder to reason about
```

This becomes especially dangerous with:

```
multiple threads

concurrent requests

singletons

static mutable fields
```

Immutability Helps

Immutable objects reduce accidental shared-state changes.

Example:

```
public sealed record GiftConfiguration(
    decimal MaximumAmount,
    string Currency);
```

If the design treats configuration as immutable:

```
many references can safely read it
```

without worrying that one consumer changes it unexpectedly.

Practical Exercise — Value Copy

Predict the output:

```
int first = 10;

int second = first;

second = 50;

Console.WriteLine(first);
Console.WriteLine(second);
```

Answer:

```
10
50
```

Reason:

```
int is value type
    →
value copied
```

Practical Exercise — Reference Copy

```
Gift first =
    new Gift
    {
        TotalAmount =
            1000m
    };

Gift second =
    first;

second.TotalAmount =
    5000m;

Console.WriteLine(
    first.TotalAmount);
```

Answer:

```
5000
```

because both references identify the same object.

Practical Exercise — Reference Reassignment

```
Gift first =
    new Gift
    {
        TotalAmount =
            1000m
    };

Gift second =
    first;

second =
    new Gift
    {
        TotalAmount =
            9000m
    };

Console.WriteLine(
    first.TotalAmount);

Console.WriteLine(
    second.TotalAmount);
```

Answer:

```
1000
9000
```

because `second` was redirected to a new object.

Practical Exercise — Array Reference

```
int[] first =
{
    1,
    2,
    3
};

int[] second =
    first;

second[0] =
    100;

Console.WriteLine(
    first[0]);
```

Answer:

```
100
```

because arrays are reference types.

Practical Exercise — String

```
string first =
    "Gift";

string second =
    first;

second +=
    " Processing";

Console.WriteLine(first);
Console.WriteLine(second);
```

Answer:

```
Gift
Gift Processing
```

because strings are immutable.

Practical Exercise — Boxing

```
int number =
    10;

object boxed =
    number;

number =
    50;

Console.WriteLine(
    boxed);
```

Answer:

```
10
```

because boxing copied the original `int` value.

Practical Exercise — Valid Unboxing

```
object boxed =
    100;

int number =
    (int)boxed;
```

Result:

```
100
```

Practical Exercise — Invalid Unboxing

```
object boxed =
    100;

long number =
    (long)boxed;
```

What happens?

Answer:

```
InvalidCastException
```

because the boxed runtime value is:

```
int
```

not:

```
long
```

Practical Exercise — Correct Numeric Conversion After Unboxing

```
object boxed =
    100;

long number =
    (long)(int)boxed;
```

Flow:

```
object
    →
unbox int
    →
convert int to long
```

Result:

```
100
```

Practical Exercise — Struct Copy

```
GiftSummary first =
    new GiftSummary
    {
        Count = 5,
        Total = 1000m
    };

GiftSummary second =
    first;

second.Total =
    5000m;
```

Expected:

```
first.Total  = 1000
second.Total = 5000
```

Practical Exercise — Struct with Reference

Suppose:

```
public struct Batch
{
    public List<long> Ids
    {
        get;
        set;
    }
}
```

Then:

```
Batch first =
    new Batch
    {
        Ids =
            new List<long>
            {
                1
            }
    };

Batch second =
    first;

second.Ids.Add(
    2);
```

What does:

```
first.Ids.Count
```

return?

Answer:

```
2
```

because copying the struct copied the reference to the same list object.

Interview Questions

What is the difference between value types and reference types in C#?

A value-type variable directly contains its value.

A reference-type variable contains a reference to an object.

With normal assignment, value types copy their value, while reference types copy the reference. This allows multiple reference variables to point to the same object.

Give examples of value types.

Examples include:

```
int
long
decimal
bool
char
DateTime
Guid
struct
enum
record struct
```

Give examples of reference types.

Examples include:

```
class
record class
string
object
dynamic
array
interface
delegate
```

Microsoft's current C# reference lists classes, interfaces, arrays, delegates, `dynamic`, `object`, and `string` as reference types.

What happens when you assign one value-type variable to another?

The value is copied.

Changing one variable afterward does not normally affect the other.

Example:

```
int a = 10;
int b = a;

b = 20;
```

`a` remains `10`.

What happens when you assign one reference-type variable to another?

The reference is copied.

Both variables can therefore point to the same object.

Changing that object's state through one reference is visible through the other reference.

Does assigning one class variable to another clone the object?

No.

It normally copies the reference.

Example:

```
Gift second =
    first;
```

does not create a separate `Gift`.

What is the difference between mutating an object and reassigning a reference?

Mutation:

```
gift.Amount = 1000m;
```

changes the object.

Reassignment:

```
gift = new Gift();
```

changes which object the variable references.

Are reference types passed by reference to methods by default?

No.

Arguments are passed by value by default.

For a reference type, the **reference value is copied**.

The copied reference still points to the same object, which is why object mutations are visible to the caller.

Are value types always stored on the stack?

No.

That is an oversimplification.

Value/reference classification primarily defines semantics. Value types can be locals, fields inside heap objects, array elements, or parts of other values. Their actual physical storage depends on context and runtime optimization.

Are reference types always stored on the heap?

The referenced object is generally managed as an object by the CLR, but a reference variable itself may exist as a local, field, array element, or other runtime storage.

Therefore saying “reference variable always lives on the heap” is also inaccurate.

What is boxing?

Boxing converts a value type to `object` or to an interface type implemented by that value type.

The CLR creates an object containing a copy of the value and provides a reference to it.

Example:

```
int value = 10;

object boxed =
    value;
```

Is boxing implicit or explicit?

Boxing is generally an implicit conversion.

Example:

```
object value =
    10;
```

The compiler performs the boxing conversion automatically.

What is unboxing?

Unboxing extracts the original value type from a boxed object.

Example:

```
object value =
    10;

int result =
    (int)value;
```

Unboxing requires an explicit conversion.

Why is unboxing explicit?

Because an `object` can refer to many different runtime types.

The runtime must verify that the object contains the expected boxed value type before extracting it.

What happens if you unbox to the wrong type?

An:

```
InvalidCastException
```

is thrown.

Example:

```
object value =
    10;

short result =
    (short)value;
```

fails because the boxed type is `int`, not `short`.

Can a boxed int be directly unboxed as long?

No.

This fails:

```
object value =
    10;

long result =
    (long)value;
```

because the boxed value is an `int`.

Instead:

```
long result =
    (long)(int)value;
```

first unboxes to `int`, then performs numeric conversion.

What happens when null is unboxed to a non-nullable value type?

The operation fails because there is no value to extract. Microsoft documents unboxing a null reference to a value type as resulting in `NullReferenceException`.

Does boxing create a copy?

Yes.

Example:

```
int number = 10;

object boxed =
    number;

number = 20;
```

The boxed object still contains the copied value `10`.

Can boxing occur when converting to an interface?

Yes.

A value type converted to an interface it implements can require boxing.

Example:

```
int value = 10;

IComparable comparable =
    value;
```

Why can boxing hurt performance?

Boxing generally requires:

```
managed allocation
+
copying the value
+
later garbage-collection work
```

Unboxing also requires runtime type checking and extracting the value. Microsoft notes that boxing/unboxing are more expensive than simple assignment.

How do generics help avoid boxing?

Generic APIs preserve the concrete value type.

Example:

```
List<int>
```

stores integers as integers rather than forcing every element through `object`.

This improves type safety and commonly avoids boxing required by older object-based APIs.

Is string a value type?

No.

`string` is a reference type.

However, strings are immutable and define value-oriented equality behavior, which sometimes makes them appear similar to value types.

Are arrays value types?

No.

Arrays are reference types.

Assigning an array variable to another variable copies the array reference, not all array elements.

Is a struct a value type?

Yes.

A `struct` defines a value type.

Assignment normally copies the struct value.

Is a record a value type or reference type?

It depends.

```
record / record class
    →
reference type

record struct
    →
value type
```

Is an enum a value type?

Yes.

Enum types are value types.

Can a struct contain a reference type?

Yes.

For example:

```
struct Example
{
    public List<int> Values;
}
```

Copying the struct copies the list reference, not the list object itself.

Why are mutable structs often discouraged?

Because value copying can produce surprising behavior.

A mutation may affect only a copied struct rather than another value from which it was copied.

Small immutable structs are generally easier to reason about.

When should I choose a class instead of a struct?

Use a class when the concept:

```
has identity

has shared mutable state

has a substantial lifecycle

is relatively large

needs inheritance
```

For example, a Fundraising `Gift` entity naturally fits class semantics.

When should I consider a struct?

Consider a struct for a relatively small value-like concept where identity is not important and copying the value is natural.

Examples could include:

```
small measurements

coordinates

small immutable domain values
```

What is the difference between ReferenceEquals and Equals?

`ReferenceEquals` answers:

```
Do these two references point to exactly the same object?
```

`Equals` can represent value equality depending on the type's implementation.

We will study equality deeply later.

Best Practices

Understand value/reference semantics before trying to memorize stack/heap details.

Do not say:

```
value type always stack
reference type always heap
```

as an absolute rule.

Remember:

```
value assignment
    →
copies value

reference assignment
    →
copies reference
```

Do not expect class assignment to clone objects.

Use immutable value types when practical.

Avoid large mutable structs unless there is a strong reason.

Prefer classes for entities with identity and lifecycle.

Prefer strongly typed generic collections such as:

```
List<int>
```

instead of object-based storage when possible.

Avoid unnecessary boxing in performance-sensitive code.

Do not optimize isolated boxing operations without measurement.

When unboxing, use the exact boxed value type.

Use pattern matching when runtime object type is uncertain.

Remember that structs can contain references, so struct assignment is not automatically a deep copy.

Treat shared mutable reference objects carefully in singleton and concurrent scenarios.

Use profiling and benchmarks for real memory/performance decisions.

Quick Revision

Concept	Meaning
Value type	Variable directly contains value
Reference type	Variable contains reference to object
Value assignment	Copies value
Reference assignment	Copies reference
`struct`	User-defined value type
`class`	Reference type
`enum`	Value type
Array	Reference type
`string`	Immutable reference type
`object`	Root reference type of unified type system
Object mutation	Change state of referenced object
Reference reassignment	Point variable at another object
Boxing	Value type → `object`/interface reference
Unboxing	Boxed object → value type
Boxing allocation	Creates managed object containing copied value
`InvalidCastException`	Possible when unboxing wrong type
Stack	Thread method-call/execution storage concept
Managed heap	Runtime-managed object storage
Shallow copy	Copies references to nested objects
Deep copy	Creates independent nested object copies
Mutable struct	Value type whose state can change
Immutable struct	Value type designed not to mutate
`ReferenceEquals`	Tests exact object identity

Complete Fundraising Example

```
public sealed class Gift
{
    public long GiftId
    {
        get;
        init;
    }

    public decimal TotalAmount
    {
        get;
        set;
    }
}

public readonly record struct Money(
    decimal Amount,
    string Currency);

// Reference-type example.

Gift gift1 =
    new Gift
    {
        GiftId = 1001,
        TotalAmount = 1000m
    };

Gift gift2 =
    gift1;

gift2.TotalAmount =
    1500m;

Console.WriteLine(
    gift1.TotalAmount);

// 1500 because both references
// point to the same Gift object.


// Value-type example.

Money amount1 =
    new Money(
        1000m,
        "USD");

Money amount2 =
    amount1;

amount2 =
    new Money(
        1500m,
        "USD");

Console.WriteLine(
    amount1.Amount);
```

```
// 1000 because Money is a value type
// and amount2 received a value copy.


// Boxing example.

int giftCount =
    25;

object boxedCount =
    giftCount;

giftCount =
    100;

Console.WriteLine(
    boxedCount);

// 25 because boxing copied the original value.


// Unboxing.

int restoredCount =
    (int)boxedCount;

Console.WriteLine(
    restoredCount);

// 25
```

Complete Concept Flow

```
C# Variable
    →
What Kind of Type?
    →
Value Type
or
Reference Type
```

Value type:

```
Value Variable
    →
Contains Value
    →
Assignment
    →
Copy Value
    →
Independent Copy
```

Reference type:

```
Reference Variable
    →
Contains Reference
    →
Reference Points to Object
    →
Assignment
    →
Copy Reference
    →
Multiple Variables Can Share Same Object
```

Mutation:

```
Shared Object
    →
Modify Through One Reference
    →
Other References See Modified Object
```

Reassignment:

```
Reference Variable
    →
Assign New Object
    →
Only That Variable Changes Target
```

Boxing:

```
Value Type
    →
Convert to object/interface
    →
Allocate Managed Object
    →
Copy Value Into Object
    →
Reference Returned
```

Unboxing:

```
Object Reference
    →
Check Boxed Runtime Type
    →
Correct?
    →
Yes
    →
Copy Value Out

No
    →
InvalidCastException
```

Fundraising view:

```
Gift
    →
Entity with identity
    →
Class
    →
Reference semantics

Money
    →
Value concept
    →
Possible immutable struct/record struct
    →
Value semantics

GiftCount int
    →
Value type

GiftCount converted to object
    →
Boxing

Object converted back to int
    →
Unboxing
```

The most important lesson from this topic is:

> **Value type versus reference type is primarily about how values behave when assigned, passed, and shared—not a simplistic rule that says “value types are on the stack and reference types are on the heap.”**

Once this is clear, later topics such as nullability, OOP, records, generics, collections, dependency injection, EF Core tracking, concurrency, and memory management become much easier to understand.

Next roadmap topic: Nullability and Null Handling.